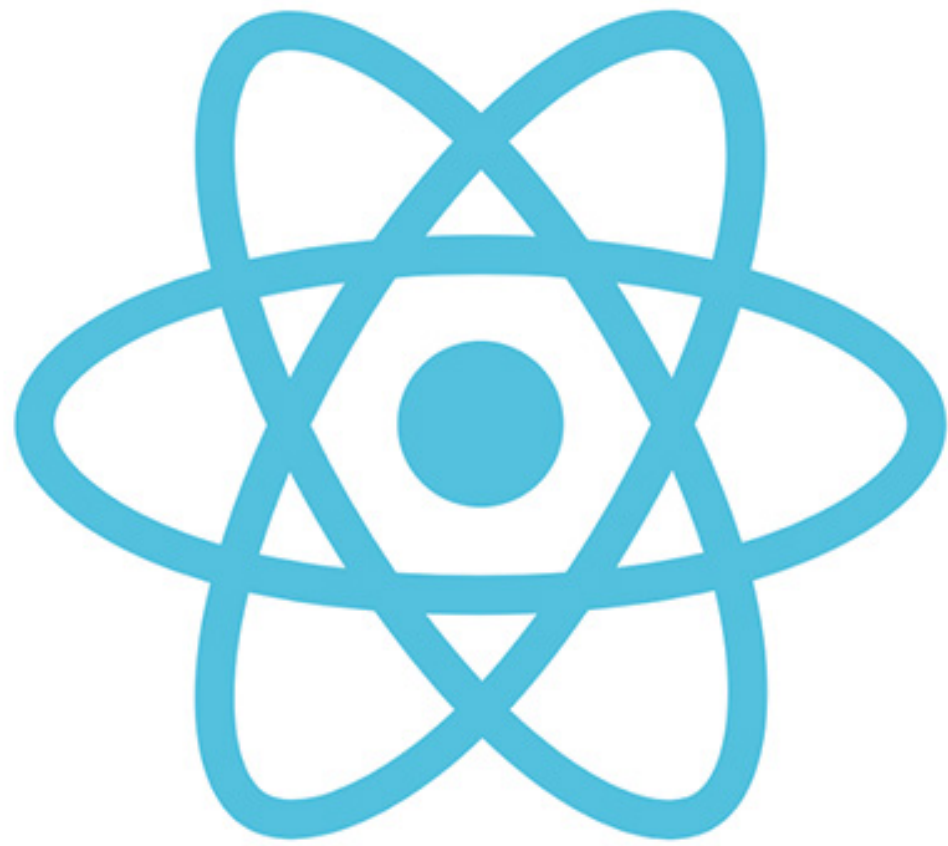




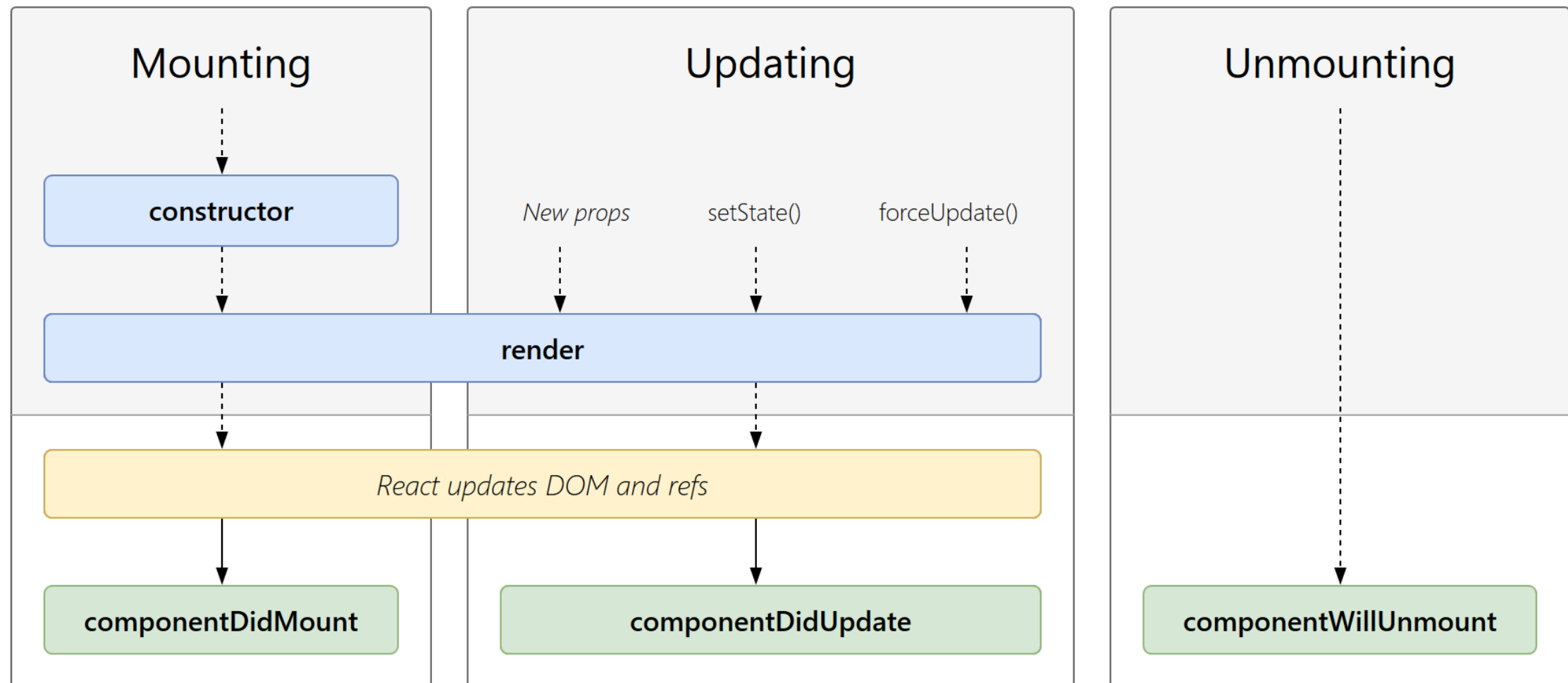
React JS

Hook

Hook

- 혹은 함수형 컴포넌트 내부에서 상태 관리나 라이프사이클 관리 같은 중요한 기능을 사용할 수 있는 일종의 연결 역할
- 혹은 사용하여 특정 기능을 함수 컴포넌트 안에 연결(hook)
- 클래스형 컴포넌트에서만 사용할 수 있었던 여러 기능을 함수형 컴포넌트에서도 손쉽게 활용할 수 있게 해줌
- 클래스의 기능을 사용하기 위한 단계를 넘어 역할이 확장
- React 내부 기능을 컴포넌트에서 쓰기 위한 공식 통로

React Component의 Lifecycle



Class Component Lifecycle

```
import { Component } from 'react';
```

```
type Props = {  
  count: number;  
};
```

```
type State = {  
  count: number;  
};
```

```
export default class CCounter extends Component<Props, State> {  
  constructor(props: Props) {  
    super(props);  
    this.state = {  
      count: props.count,  
    };  
  }  
}
```

=> 계속

Class Component Lifecycle

=> 앞에서 계속

```
handleClick = (): void => {  
  this.setState((prevState) => {  
    return { count: prevState.count + 1 };  
  });  
};
```

```
render() {  
  console.log('render 호출');  
  
  return (  
    <div>  
      <p>{this.state.count} 번 클릭했습니다</p>  
      <button onClick={this.handleClick}>클릭</button>  
    </div>  
  );  
}
```

=> 계속

Class Component Lifecycle

=> 앞에서 계속

```
componentDidMount(): void {  
  console.log('componentDidMount 호출');  
}
```

```
componentDidUpdate(): void {  
  console.log('componentDidUpdate 호출');  
}
```

```
componentWillUnmount(): void {  
  console.log('componentWillUnmount 호출');  
}  
}
```


useState - 내부동작

```
let hooks = [];  
let currentIndex = 0;  
  
function useState(initialValue) {  
  const index = currentIndex;  
  
  if (hooks[index] === undefined) {  
    hooks[index] = initialValue;  
  }  
  
  function setState(newValue) {  
    hooks[index] = newValue;  
    render(); // 다시 렌더링  
  }  
  
  currentIndex++;  
  return [hooks[index], setState];  
}
```


useEffect

- componentDidMount, componentDidUpdate 기능을 통합한 역할
- 의존성 배열에 기술된 변수들이 변경되었을때 useEffect가 실행
- 의존성 배열이 빈 배열일때는 최초 한번만 실행, 생략시 랜더링 할때마다 실행

```
import { useEffect, useState } from 'react';
export default function ExUseEffect() {
  const [count, setCount] = useState<number>(0);
  useEffect(() => {
    document.title = `${count}번 클릭했습니다`;
  }, [count]); // 의존성배열, 변경되었을때 useEffect가 실행되어야하는 변수들
               // [](빈배열)일 경우 최초 한번만, 생략할때는 랜더링될때마다 실행
  return (
    <div>
      <p>{count}번 클릭했습니다</p>
      <button onClick={() => setCount((prev)=>prev + 1)}>클릭</button>
    </div>
  );
}
```

useEffect

```
interface User { id: number; name: string; email: string; }

export default function UserList() {
  const [users, setUsers] = useState<User[]>([]);
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((res) => res.json())
      .then((data: User[]) => setUsers(data));
  }, []);
  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name} ({user.email})</li>
      ))}
    </ul>
  );
}
```

useEffect-async/await

```
export default function UserList() {  
  const [users, setUsers] = useState<User[]>([]);  
  const [error, setError] = useState<string | null>(null);  
  useEffect(() => {  
    const load = async () => {  
      try {  
        const res = await fetch('https://jsonplaceholder.typicode.com/users');  
        if (!res.ok) {  
          throw new Error('요청 실패');  
        }  
        const data: User[] = await res.json();  
        setUsers(data);  
      } catch (err) {  
        if (err instanceof Error) { setError(err.message); }  
      }  
    };  
    load();  
  }, []);  
  => 계속
```

useEffect-async/await

=> 앞에서 계속

```
if (error) return <p>{error}</p>;

return (
  <ul>
    {users.map((user) => (
      <li key={user.id}>
        {user.name} ({user.email})
      </li>
    ))}
  </ul>
);
}
```

useEffect-의존성 배열

```
import { useEffect, useState } from 'react';
export default function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log('count 변경됨:', count);
  }, [count]); // 의존성 배열에 있는 속성이 바뀔 때마다 실행

  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

useEffect

```
import { useEffect, useState } from 'react';

export default function Test() {
  const [time, setTime] = useState<number>(0);
  useEffect(() => {
    console.log('타이머 시작');
    const id = setInterval(() => {
      setTime((t) => t + 1);
    }, 1000);
    //cleanup
    return () => {
      console.log('타이머 정리');
      clearInterval(id);
    };
  }, []);
  return <h1>{time}초</h1>;
}
```

useEffect-영화정보

<https://raw.githubusercontent.com/wizard113/ML-Basic/main/movie.json>



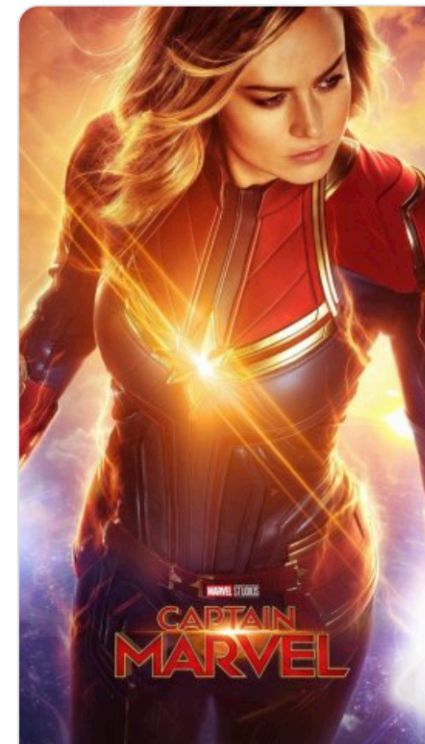
프로젝트 헤일메리



제이슨 본



로건



캡틴 마블



올레

useMemo

- 성능 최적화를 위해 사용
- 값이 변경되지 않았다면 이전에 계산된 결과를 재사용하게 해줌

```
import { useMemo } from 'react';  
interface Props {  
  num: number;  
}  
export default function ExUseMemo({ num }: Props) {  
  const expensiveCalc = (n: number) => {  
    console.log('비싼 계산 실행 중...');  
    return num * 2;  
  };  
  
  const memoizedValue = useMemo(() => expensiveCalc(num), [num]);  
  
  return <div>결과: {memoizedValue}</div>;  
}
```


useMemo

```
import { useMemo, useState } from 'react';

export default function Test() {
  const [query, setQuery] = useState('');
  const [count, setCount] = useState(0);
  const list = ['apple', 'banana', 'grape', 'orange'];

  const filtered = useMemo(() => {
    console.log('필터 실행됨');
    return list.filter((item) => item.includes(query));
  }, [query]); //query만 의존
```

=> 계속

useMemo

```
return (  
  <div>  
    <h3>검색</h3>  
    <input  
      value={query}  
      onChange={(e) => setQuery(e.target.value)}  
      placeholder="검색어 입력"  
    />  
    <h3>관련 없는 상태 변경</h3>  
    <button onClick={() => setCount(count + 1)}>count 증가 ({count})</button>  
    <ul>  
      {filtered.map((item) => (  
        <li key={item}>{item}</li>  
      ))}  
    </ul>  
  </div>  
}
```

useCallback

- 함수가 다시 생성되지 않도록 함수를 메모이제이션
- 자식 컴포넌트로 전달될 때 불필요하게 재 생성되지 않도록 함

```
import { useCallback, useState } from 'react';
```

```
function ParentComponent() {  
  const [count, setCount] = useState(0);
```

```
  const handleClick = useCallback(() => {  
    console.log('버튼 클릭');  
  }, []);
```

```
  return (  
    <div>  
      <button onClick={handleClick}>클릭</button>  
      <button onClick={() => setCount(count + 1)}>카운트 증가</button>  
    </div>  
  );  
}
```

useRef

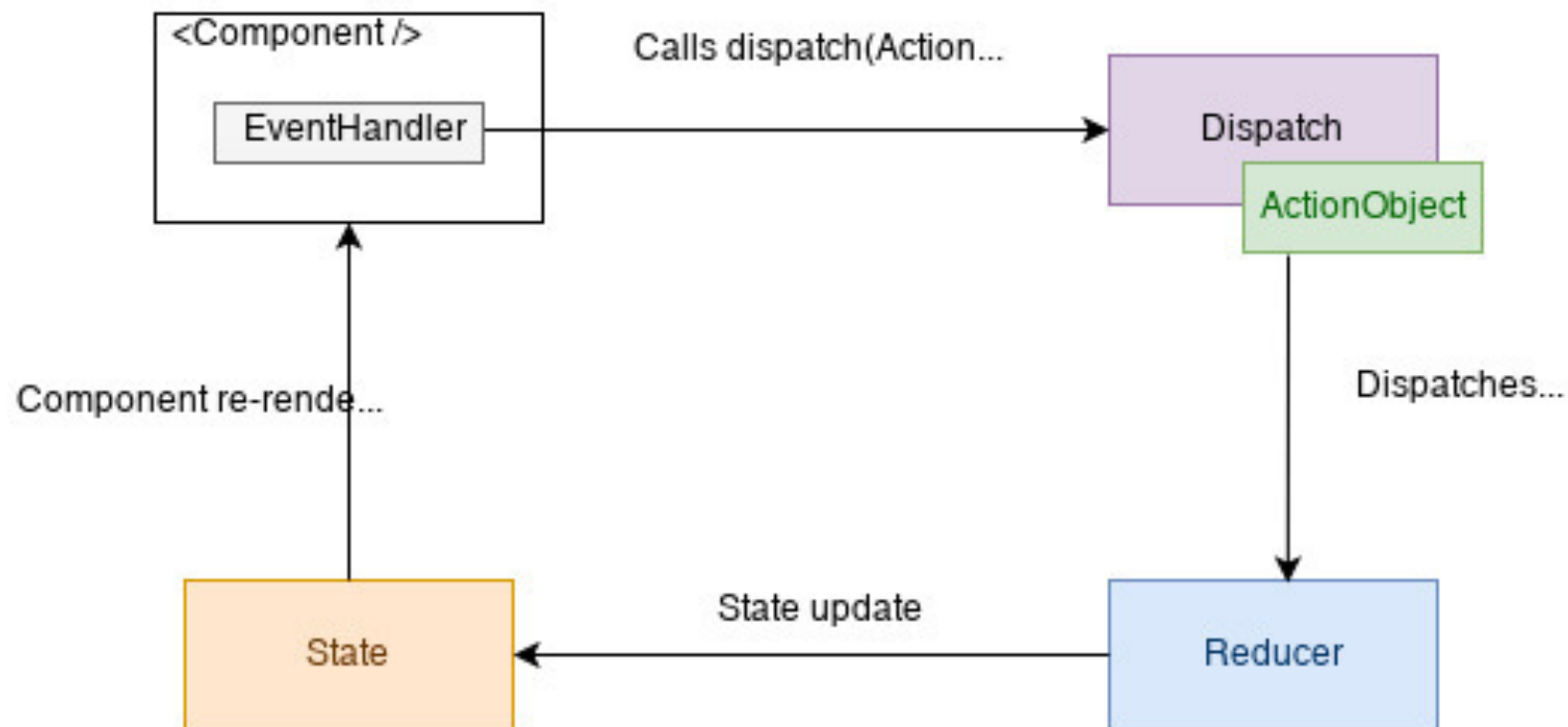
- DOM 접근 및 리렌더링 없는 데이터 저장
- DOM 요소에 직접 접근할때 얻어진 참조값을 기억
- useRef에 담긴 값은 변경되어도 컴포넌트를 리렌더링하지 않음

```
import { useRef } from 'react';
export default function Test() {
  const inputRef = useRef<HTMLInputElement>(null); // DOM 접근용
  const clickCountRef = useRef<number>(0); // 표시할 필요없는값 저장용
  const handleClick = () => {
    inputRef.current?.focus();
    clickCountRef.current += 1;
    console.log(`클릭 횟수: ${clickCountRef.current}`);
  };
  return (
    <div>
      <input ref={inputRef} type="text" placeholder="입력하세요" />
      <button onClick={handleClick}>포커스 및 카운트 증가</button>
    </div>
  );
}
```

useReducer

- 복잡한 상태 업데이트 로직을 효율적이고 종합적으로 관리
- 상태와 변경 로직을 분리해서 관리하는 목적
- dispatch로 액션을 보내고 reducer가 새로운 state를 반환
- 복잡한 상태나 조건이 많을 때 useState보다 유리

useReducer()



Viewer does not support full SVG 1.1

useReducer

```
import { useReducer } from 'react';

type Action = 'INC' | 'DEC';
// type Action = { type: 'INC' } | { type: 'DEC' };

function countReducer(current: number, action: Action): number {
  if (action === 'INC') {
    return current + 1;
  } else if (action === 'DEC') {
    return current - 1;
  }
  return current;
}
```

=> 다음에 계속

useReducer

```
export default function Counter() {  
  const [count, dispatch] = useReducer(countReducer, 0);  
  
  function increase(): void {  
    dispatch('INC');  
  }  
  
  function decrease(): void {  
    dispatch('DEC');  
  }  
  
  return (  
    <div>  
      <input type="button" value="-" onClick={decrease} />  
      <input type="button" value="+" onClick={increase} />  
      <span>{count}</span>  
    </div>  
  );  
}
```


useReducer

```
import { useReducer } from 'react';
type State = { email: string; password: string };
type Action =
  | { type: 'CHANGE_EMAIL'; value: string }
  | { type: 'CHANGE_PASSWORD'; value: string }
  | { type: 'RESET' };

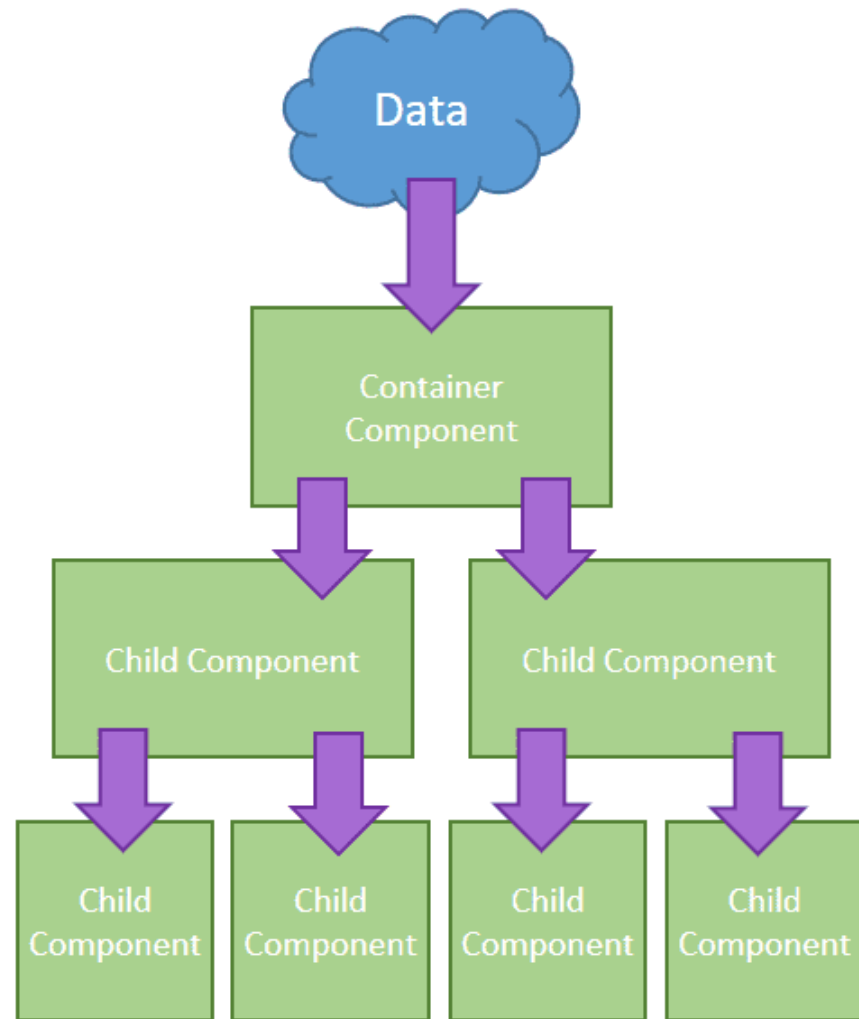
function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'CHANGE_EMAIL':
      return { ...state, email: action.value };
    case 'CHANGE_PASSWORD':
      return { ...state, password: action.value };
    case 'RESET':
      return { email: '', password: '' };
    default: return state;
  }
}
```

=> 계속

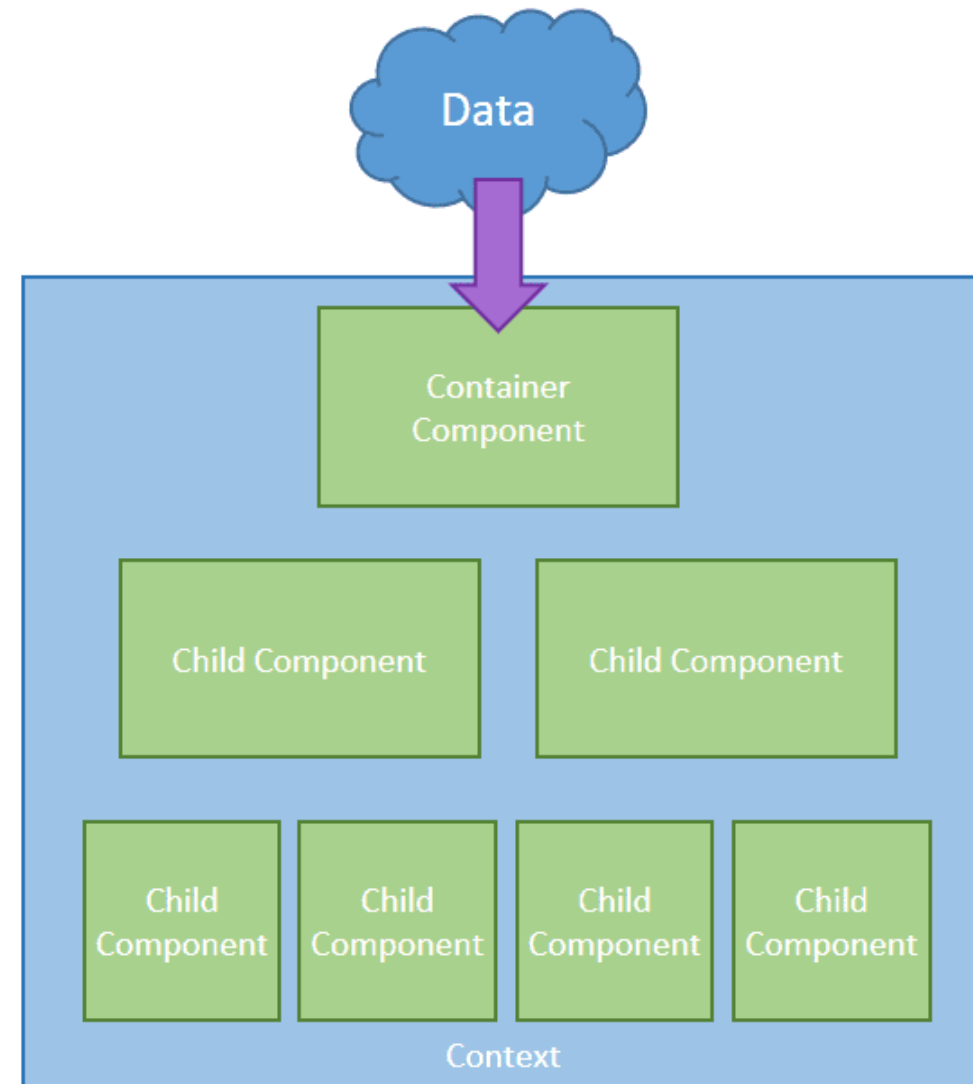
useReducer

```
export default function Login() {  
  const [state, dispatch] = useReducer(reducer, {  
    email: '',  
    password: '',  
  });  
  return (  
    <div>  
      <input placeholder="email" value={state.email}  
        onChange={(e) => dispatch({ type: 'CHANGE_EMAIL', value: e.target.value })}/>  
      <input placeholder="password" value={state.password}  
        onChange={(e) => dispatch({ type: 'CHANGE_PASSWORD', value: e.target.value })}/>  
      <button onClick={() => dispatch({ type: 'RESET' })}>reset</button>  
      <p>{state.email}</p>  
      <p>{state.password}</p>  
    </div>  
  );  
}
```

Context



prop drilling



context API

Context 생성

```
import { createContext } from 'react';

type Theme = 'light' | 'dark';

type ThemeContextType = {
  theme: Theme;
  toggleTheme: () => void;
};

const ThemeContext = createContext<ThemeContextType>({
  theme: 'light',
  toggleTheme: () => {
    console.log();
  },
});

export default ThemeContext;
```

Context-Provider

```
import { useState } from 'react';
import ThemeContext from './ThemeContext';
import MainContent from './Main';

type Theme = 'light' | 'dark';
export default function DarkOrLight() {
  const [theme, setTheme] = useState<Theme>('light');

  const toggleTheme = (): void => {
    setTheme((prev) => (prev === 'light' ? 'dark' : 'light'));
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      <MainContent />
    </ThemeContext.Provider>
  );
}
```

Context-useContext

```
import { useContext } from 'react';
import ThemeContext from './ThemeContext';
export default function MainContext() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  const styles: React.CSSProperties = {
    padding: '20px',
    margin: '20px',
    textAlign: 'center',
    backgroundColor: theme === 'light' ? '#fff' : '#333',
    color: theme === 'light' ? '#000' : '#fff',
    border: theme === 'light' ? '1px solid #000' : '1px solid #fff',
  };
  return (
    <div style={styles}>
      <p>현재 테마: {theme}</p>
      <button onClick={toggleTheme}>테마 변경</button>
    </div>
  );
}
```

Custom Hook

- 여러 Hook을 묶어서 재사용하는 함수

```
import { useState } from 'react';
```

```
export function useToggle(initial: boolean = false) {  
  const [value, setValue] = useState<boolean>(initial);
```

```
  const toggle = () => {  
    setValue((prev) => !prev);  
  };  
  
  return { value, toggle };  
}
```

```
import { useToggle } from "../useToggle";  
  
export default function Test() {  
  const { value, toggle } = useToggle();  
  
  return (  
    <div>  
      <p>{value ? "ON" : "OFF"}</p>  
      <button onClick={toggle}>toggle</button>  
    </div>  
  );  
}
```

Custom Hook - useFetch

```
import { useEffect, useState } from 'react';

export function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    const load = async () => {
      const res = await fetch(url);
      const json = await res.json();
      setData(json);
      setLoading(false);
    };

    load();
  }, [url]);

  return { data, loading };
}
```


Custom Hook - useFetch

```
import { useFetch } from './useFetch';
type Movie = {
  id: number;
  title: string;
};

export default function Test() {
  const { data, loading } = useFetch<{ movies: Movie[] }>(
    'https://raw.githubusercontent.com/wizard113/ML-Basic/main/movie.json',
  );
  if (loading) return <p>loading...</p>;
  return (
    <ul>
      {data?.movies.map((m) => (
        <li key={m.id}>{m.title}</li>
      ))}
    </ul>
  );
}
```


실습 - ToDoList

오늘은  17

Sun Apr 26 2026

새로운 Todo 작성하기 

새로운 Todo...

추가

Todo List 

검색어를 입력하세요

- | | | | |
|--------------------------|------------|--------------|----|
| ☐ | React 공부하기 | 2026. 4. 26. | 삭제 |
| ☐ | 빨래 널기 | 2026. 4. 26. | 삭제 |
| ☐ | 노래 연습하기 | 2026. 4. 26. | 삭제 |